

Rinku: Self-Provable URLs for Trustless Verification

Abstract. We propose a distributed ledger in which URLs serve as self-contained cryptographic proofs. This would enable trustless verification without reliance on external infrastructure or services. A Rinku URL carries not just transaction data, but its complete verification path - ancestry, signatures, and checkpoint anchors. Ultimately, *the link itself is the proof*. This paper focuses on the URL-native proof system that a distributed network could incorporate.

1. The Problem with Verification Today

Traditional blockchain networks require infrastructure or extraneous software for verification:

1. **Node dependency** - Users must trust a node operator or run their own
2. **State opacity** - Verification requires querying external systems
3. **Proof complexity** - Light clients need specific tooling and trusted endpoints

Even with light clients, users ultimately must trust *someone else's infrastructure* to provide proofs. The verification is outsourced, not self-contained.

What if the URL itself could prove its validity?

2. URLs as Proofs

Instead of storing data on-chain and fetching proofs from nodes, we encode proofs directly into URLs:

e.g

```
rinku://tx/{base64url(deflate(transaction + ancestry + checkpoint))}
```

A Rinku URL contains:

- The transaction data (sender, recipient, amount, signature)
- Ancestry chain back to a finalized checkpoint
- Checkpoint anchor (Merkle root, validator attestations)

3. How It Works

3.1 Transaction Encoding

Transactions are encoded as compressed JSON directly in the URL path:

Transaction -> JSON -> DEFLATE -> Base64url -> URL

A single transaction URL is roughly 600 characters. With 5 levels of ancestry (proving the transaction chains back to a checkpoint), URLs remain under 1,500 characters which could fit within a QR code.

3.2 Proof Structure

A proof bundle contains:

```
{
  "tx": {
    "from": "a1b2c3...",
    "to": "d4e5f6...",
    "amount": 1000000,
    "nonce": 42,
    "sig": "..."
  },
  "fromPubKey": "BASE64_ECDSA_P256_PUBLIC_KEY",
  "hash": "sha256(tx)",
  "parents": [/* recursive proof bundles */],
  "checkpoint": {
    "id": "cp_789",
    "merkleRoot": "...",
    "height": 1000,
    "blsAggregateSig": "...",
    "signerBitmap": "...",
    "validatorProof": { /* MerkleSumTree multi-proof */ }
  }
}
```

Key elements:

- fromPubKey - The sender's full ECDSA P-256 public key, enabling signature verification. The from field is its fingerprint (SHA-256 truncated to 40 hex chars).
- checkpoint - Contains the finality proof: BLS aggregated signature from validators, a bitmap indicating which validators signed, and a Merkle proof for the validator set.

Each parent is itself a proof bundle, creating a recursive structure that traces back to a known checkpoint.

3.3 Verification Algorithm

```
function verify(proofUrl):
  bundle = decode(proofUrl)

  // 1. Verify public key matches fingerprint
  assert fingerprint(bundle.fromPubKey) == bundle.tx.from

  // 2. Verify transaction signature via the public key
  assert ecdsaVerify(bundle.fromPubKey, bundle.tx, bundle.tx.sig)

  // 3. Verify hash integrity
  assert sha256(bundle.tx) == bundle.hash

  // 4. Verify all parents
  for parent in bundle.parents:
    assert verify(parent)

  // 5. Verify checkpoint finality
  assert verifyCheckpointSignatures(bundle.checkpoint)

  return true
```

3.4 Data Availability

Important clarification: Self-provable URLs guarantee *verification* without infrastructure, not *discovery*. The URL must reach the verifier through some transport:

- Sender shares URL via QR code, message, email etc
- URLs are stored in application databases
- Crawlers can follow parent references to reconstruct comprehensive history

Once the verifier has the URL, no further dependancy on infrastructure is needed. The proof is discrete. This is analogous to a signed document: the signature proves authenticity, but the document must still be delivered.

3.5 The Rinku Network

Rinku is a DAG-based distributed ledger where:

- **Account micro-chains** - Each account maintains its own chain of transactions, with each new transaction referencing the account's previous transaction.
- **DAG linking** - Transactions also reference 1-2 recent "tip" transactions from other accounts, weaving individual chains into a shared DAG structure.
- **Weight-based ordering** - Conflicts are resolved by cumulative weight, providing Sybil resistance without proof-of-work.

Finality via checkpoints:

Validators periodically create checkpoints that finalize batches of transactions. A checkpoint includes:

- A Merkle root committing to all finalized transactions
- A BLS-aggregated signature from $\geq 67\%$ of the validator set
- A commitment to the next validator set

Once a transaction is included in a checkpoint, a self-provable URL can be generated. The checkpoint's aggregated signature becomes the trust anchor embedded in the proof.

For complete protocol details including validator selection, staking, and gas mechanics, see the full technical specification.

4. Proof Profiles

Different use cases require different security/size tradeoffs:

Profile A: Receipt (~600 - 2,300 characters)

What it proves: Transaction is valid and chains to a prior checkpoint **Trust assumption:** Verifier trusts the checkpoint was correctly signed **Use case:** PoS receipts, payment confirmations

`rinku://tx/{payload}`

Profile B: Full Finality (~3,000 - 10,000 characters)

What it proves: Transaction is Merkle-included in a checkpoint signed by $\geq 67\%$ of validators **Trust assumption:** Verifier knows the validator set **Use case:** High-value settlements

`rinku://txp/{payload}`

Profile C: Self-Contained (~1,600 - 2,800 characters)

What it proves: Everything in Profile B, in addition to the validator set commitment itself **Trust assumption:** Trust anchor is minimal and can be pinned once (genesis or pinned checkpoint); subsequent proofs are offline-verifiable **Use case:** Fully offline verification, air-gapped systems, legal evidence

`rinku://sp/{payload}`

Profile C is the most powerful - it proves finality without any external trust anchors beyond knowing which network you are verifying against.

5. Size Analysis

Real-world measurements using high-entropy data (e.g signatures):

Proof Type	Transactions	URL Length
Single tx	1	~600 chars
2-depth ancestry	3	~940 chars
5-depth ancestry	6	~1,400 chars
10-depth ancestry	11	~2,200 chars

Platform Compatibility

Platform	Limit	Single tx	5-depth
QR Code (L)	2,953 bytes	x	x
QR Code (H)	1,273 bytes	x	
Browser URL	65KB+	x	x

Single transactions and short ancestry chains fit in QR codes. Complex proofs use typical URL sharing.

note QR capacity depends on QR encoding mode; base64url typically uses byte mode. For maximum density we recommend a QR-optimized encoding (e.g base45) for tx/sp receipts.

6. Trust Bootstrapping

A fresh verifier needs a trust anchor:

1. **Genesis trust** - Know the genesis validator set
2. **Checkpoint chain** - Each checkpoint commits to the next validator set
3. **Pinned checkpoint** - Trust a recent checkpoint from an out-of-band source

This mirrors TLS certificate chains: the proof is self-contained, but root trust must be established externally. Once bootstrapped, all subsequent proofs verify offline.

7. Why This Matters

7.1 Infrastructure Independence

Traditional flow:

User -> Trust Node -> Query Proof -> Verify with Node's Help

Rinku flow:

User -> Receive URL -> Verify Locally

7.2 Portable Proofs

A Rinku URL can be:

- Printed as a QR code on a receipt
- Sent via SMS, email, or any messaging app
- Embedded in a PDF or document
- Stored offline indefinitely

The proof remains valid as long as the respective cryptography holds.

7.3 Offline-First

Verification works completely offline. This enables:

- PoS in areas with poor connectivity
- Air-gapped security systems
- Archival verification that withstands over large periods of time
- Cross-border payments without infrastructure

8. Cryptographic Primitives

The proof system uses standard, well-audited cryptography:

Component	Algorithm	Purpose
Transaction signatures	ECDSA P-256	Sender authorization
Hash function	SHA-256	Transaction identity, Merkle trees
Validator signatures	BLS12-381	Aggregated checkpoint attestation
Compression	DEFLATE	URL size reduction
Encoding	Base64url	URL-safe representation

ECDSA verification can use the Web Crypto API; BLS verification can use a WASM library (e.g blst)

9. Limitations

Size constraints: Complex proofs (30+ transactions) exceed QR capacity and require standard URL sharing.

Bootstrap requirement: First-time verifiers need a trust anchor (genesis or pinned checkpoint).

URL mutability: URLs can be shared but not modified. Proof updates require re-generated URLs.

Compression variability: Actual compression ratios depend on transaction content; high-entropy data compresses less.

10. Conclusion

Rinku demonstrates that distributed ledger proofs can be fully self-contained within URLs. By encoding transaction data, ancestry chains, and checkpoint anchors directly into the URL, we can eliminate infrastructure dependencies for verification.

The core innovation is architectural: treating URLs as the canonical proof object rather than as references to external state. This enables truly trustless, offline verification - a property no existing blockchain currently supports.

The link is the proof.

Appendix A: Encoding Specification

A.1 URL Format

`rinku://{profile}/{base64url(deflate(json))}`

Profiles:

- tx - Transaction receipt (Profile A)
- txp - Transaction with full ancestry (Profile A+)
- sp - Self-contained finality proof (Profile C)

A.2 Transaction Schema

```

interface Transaction {
  from: string; // 40-char hex fingerprint
  to: string; // 40-char hex fingerprint
  amount: number; // Smallest units (8 decimals)
  fee: number; // Gas fee
  nonce: number; // Sender sequence number
  tipUrls: string[]; // 0-2 parent references
  ts: number; // Unix timestamp (ms)
  sig: string; // ECDSA P-256 signature
}

```

A.3 Proof Bundle Schema

```

interface ProofBundle {
  tx: Transaction;
  fromPubKey: string; // Base64 ECDSA P-256 public key (SPKI format)
  hash: string; // SHA-256 hex of canonical tx JSON
  parents: ProofBundle[]; // Recursive ancestry
  checkpoint: {
    id: string;
    merkleRoot: string;
    height: number;
    blsAggregateSig: string; // BLS12-381 aggregated signature
    signerBitmap: string; // Bitmap of which validators signed
    validatorProof: { // MerkleSumTree multi-proof
      signerLeaves: Array<{ pubKey: string; weight: number }>;
      auxiliaryNodes: string[];
      totalWeight: number;
    };
  };
}

```

A.4 Canonical JSON

Transaction fields are serialized in deterministic order for consistent hashing: from, to, amount, fee, nonce, tipUrls, ts, sig. Canonical JSON = UTF-8, no whitespace, exact numeric encoding rules.

Appendix B: Reference Implementation

A minimal verifier in JavaScript:

```

async function verifyProofUrl(url) {
  // 1. Parse and decode
  const [, profile, payload] = url.match(/rinku:\V\(\w+)\V(.+)/);
  const json = inflate(base64urlDecode(payload));
  const bundle = JSON.parse(json);
}

```



```

// 2. Verify public key matches fingerprint
const pubKeyBytes = base64Decode(bundle.fromPubKey);
const fingerprint = (await sha256(pubKeyBytes)).slice(0, 40);
if (fingerprint !== bundle.tx.from) {
  throw new Error('Public key does not match sender fingerprint');
}

// 3. Verify transaction signature using bundled public key
const txBytes = canonicalize(bundle.tx);
const pubKey = await crypto.subtle.importKey(
  'spki', pubKeyBytes,
  { name: 'ECDSA', namedCurve: 'P-256' },
  false, ['verify']
);
const valid = await crypto.subtle.verify(
  { name: 'ECDSA', hash: 'SHA-256' },
  pubKey,
  base64Decode(bundle.tx.sig),
  txBytes
);
if (!valid) throw new Error('Invalid signature');

// 4. Verify hash integrity
const hash = await sha256Hex(txBytes);
if (hash !== bundle.hash) throw new Error('Hash mismatch');

// 5. Verify parents recursively
for (const parent of bundle.parents) {
  await verifyProofUrl(encodeBundle(parent));
}

// 6. Verify checkpoint finality (BLS aggregate signature)
return verifyCheckpointSignatures(bundle.checkpoint);
}

```

References

1. D. Boneh, M. Drijvers, and G. Neven. "Compact Multi-Signatures for Smaller Blockchains." ASIACRYPT 2018. <https://crypto.stanford.edu/~dabo/pubs/papers/BLSmultisig.html>
 2. D. Boneh, B. Lynn, and H. Shacham. "Short Signatures from the Weil Pairing." Journal of Cryptology, 2004.
 3. R. Merkle. "A Digital Signature Based on a Conventional Encryption Function." CRYPTO 1987.
-